

Vulnerability assessment

OpenVAS/Greenbone

Fagnani Marco, Radice Lorenzo

May 19, 2026

Contents

1	Initial configuration	2
1.1	Greenbone	2
1.2	Debian	2
1.2.1	Apache	3
1.2.2	Cups	4
1.2.3	Docker Vulnet	5
1.2.4	FTP	13
1.2.5	Samba	13
1.2.6	SSH	14
1.2.7	Syslog	14
1.3	Windows XP	15
1.4	Network configuration	15
2	System Discovery	18
2.1	Get Debian VM IP address	18
2.2	Configure Greenbone Gateway	19
2.3	Greenbone Web Interface	20
2.4	System Scan	22
3	OpenVAS Scan	27
3.1	OpenVAS Scan	27
3.1.1	Configuration → Ports	28
3.1.2	Target configuration	30
3.1.3	Starting the scan	34
4	AI Usage Declaration	35

Chapter 1

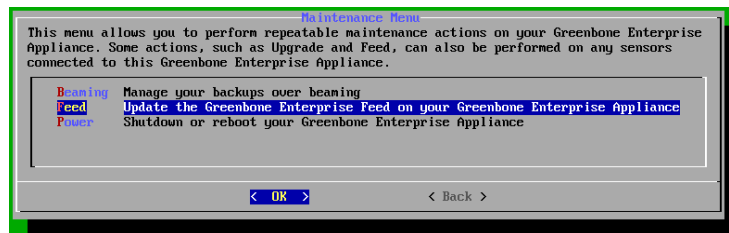
Initial configuration

1.1 Greenbone

Greenbone/OpenVAS is an open-source vulnerability scanning tool that provides comprehensive security assessments for IT systems. To set up Greenbone/OpenVAS, firstly download the official Greenbone/OpenVAS OVA file from the Greenbone website.

<https://www.greenbone.net/en/openvas-free/>

Then, import the OVA file into VirtualBox 6.1 or higher. Assign at least 5GB of RAM and 2 CPU cores to the virtual machine. It's better to use about 12GB of RAM. For the first configuration, use the bridge network adapter, which allows the virtual machine to be on the same network as the host machine. Start the Greenbone/OpenVAS virtual machine and follow the on-screen instructions to complete the initial setup. Assign a user for the web interface. Once the setup is complete, access the Greenbone/OpenVAS update the feed by going to **Maintenance>Feed>Update** and wait it to synchronize (usually hours).



1.2 Debian

Debian is an open-source operating system based on GNU/Linux. It will be used as a base to create a vulnerable machine. We used the latest stable version of Debian, which is Debian 13 Trixie.

1.2.1 Apache

Install Apache web server and create weak certificates for it. configure Apache to allow the use of weak protocols and ciphers, such as SSLv3, TLS1.1, DES, RC4 and MD5.

```
1  #!/bin/bash
2
3  apt-get update
4  apt-get install -y apache2 php libapache2-mod-php
5  rm -rf /var/lib/apt/lists/* /var/cache/apt/archives/*
6
7  a2enmod ssl
8  a2enmod rewrite
9  a2enmod headers
10 a2enmod proxy
11 a2enmod proxy_http
12 a2dissite 000-default.conf
13 echo "ServerName 127.0.0.1" | tee /etc/apache2/conf-available/servername.conf
14 a2enconf servername
15
16 output_dir=certs
17 mkdir -p $output_dir
18 cd $output_dir
19 openssl genrsa -out weak.key 1024
20 openssl req -new -key weak.key -out weak.csr -subj
21 ↪ "C=XX/ST=Lab/L=Vulnbox/O=Insecure/OU=Testing/CN=localhost"
22 openssl x509 -req -days 1 -in weak.csr -signkey weak.key -out weak.crt -sha1
23 ↪ #openssl x509 -req -days 3650 -in weak.csr -signkey weak.key -out weak_md5.crt
24 ↪ -md5
25
26 mkdir -p /etc/apache2/ssl
27 cp weak.* /etc/apache2/ssl/
28
29 tee /etc/apache2/sites-available/vulnbox.conf << EOF
30 <VirtualHost *:80>
31     ServerName localhost
32
33     ProxyPreserveHost On
34     ProxyPass / http://127.0.0.1:8000/
35     ProxyPassReverse / http://127.0.0.1:8000/
36
37     # No security headers
38 </VirtualHost>
39
40 <VirtualHost *:443>
41     ServerName localhost
42
43     SSLEngine on
44     SSLCertificateFile /etc/apache2/ssl/weak.crt
45     SSLCertificateKeyFile /etc/apache2/ssl/weak.key
46
47     # Allow TLS 1.0 only if OpenSSL supports it
48     SSLProtocol all -SSLv3 -TLSv1.3 -TLSv1.2 -TLSv1.1
49     #SSLProtocol all -SSLv3
50
51     SSLCipherSuite ALL:NULL:EXPORT:LOW:MEDIUM:DES:RC4:MD5:@SECLEVEL=0
52     #SSLCipherSuite aNULL:EXPORT:LOW:ALL:@SECLEVEL=0
53     #SSLCipherSuite HIGH:!aNULL:@SECLEVEL=0
```

```

52     #SSLCipherSuite ALL:NULL:EXPORT:LOW:@SECLEVEL=0
53
54     SSLHonorCipherOrder off
55
56     #SSLCompression on
57     SSLSessionTickets On
58
59     SSLOpenSSLConfCmd MinProtocol SSLv3
60     SSLOpenSSLConfCmd CipherString DEFAULT:@SECLEVEL=0
61
62     ProxyPreserveHost On
63     ProxyPass / http://127.0.0.1:8000/
64     ProxyPassReverse / http://127.0.0.1:8000/
65
66     Header always unset Strict-Transport-Security
67     Header always unset X-Frame-Options
68     Header always unset X-Content-Type-Options
69 </VirtualHost>
70 EOF
71
72 a2ensite vulnbox.conf
73
74 tee /etc/php/7.4/apache2/php.ini << EOF
75 display_errors = On
76 display_startup_errors = On
77 allow_url_fopen = On
78 allow_url_include = On
79 disable_functions =
80 open_basedir =
81 expose_php = On
82 EOF
83
84 systemctl restart apache2

```

Any website can be used as exposed service. In our case we used an old project of ours, properly modified to be vulnerable.

```
git clone -b vulnerable https://github.com/ozneroL541/artigianato-online.git
```

To run it execute the following commands in the terminal:

```

1 cd artigianato-online/src
2 docker compose up --remove-orphans -d

```

1.2.2 Cups

Install the CUPS printing system and configure it to allow remote administration without authentication.

```

1 #!/bin/bash
2
3 apt-get update
4 apt-get install -y cups
5 rm -rf /var/lib/apt/lists/* /var/cache/apt/archives/*
6
7 tee /etc/cups/cupsd.conf << EOF
8 Listen 0.0.0.0:631
9
10 <Location />

```

```

11     Order allow,deny
12     Allow all
13 </Location>
14
15 <Location /admin>
16     Order allow,deny
17     Allow all
18 </Location>
19
20 WebInterface Yes
21 DefaultAuthType Basic
22 EOF
23
24 systemctl enable --now cups

```

1.2.3 Docker Vulnet

To simulate a complex network environment, we are using a docker container with multiple vulnerable services. The scope of this network is to show the capabilities of Greenbone/OpenVAS in scanning a complex system. To set up the network we used the following docker compose file:

```

1 #####
2 # Vulnerability Assessment Lab
3 #
4 # Network: 172.30.0.0/22
5 #   host      - 172.30.0.1/24
6 #   vulnbox1 - 172.30.0.2/24 172.30.1.1/24
7 #   vulnbox2 - 172.30.1.2/24 172.30.2.1/24
8 #   vulnbox3 - 172.30.2.2/24
9 #   vulnbox4 - 172.30.2.3/24
10 #   vulnbox5 - 172.30.2.4/24 172.30.3.1/24
11 #   vulnbox6 - 172.30.3.2/24
12 #####
13
14 networks:
15   # Bridge { directly reachable from host
16   net0:
17     driver: bridge
18     driver_opts:
19       com.docker.network.bridge.name: "vulnlab0"
20     ipam:
21       config:
22         - subnet: 172.30.0.0/24
23           gateway: 172.30.0.1
24
25   # Internal macvlan segments { isolated from host, routed only through vulnbox1
26   net1:
27     driver: bridge
28     driver_opts:
29       com.docker.network.bridge.name: "vulnlab1"
30     internal: true           # blocks direct host - net1 traffic
31     ipam:
32       config:
33         - subnet: 172.30.1.0/24
34           gateway: 172.30.1.254
35

```

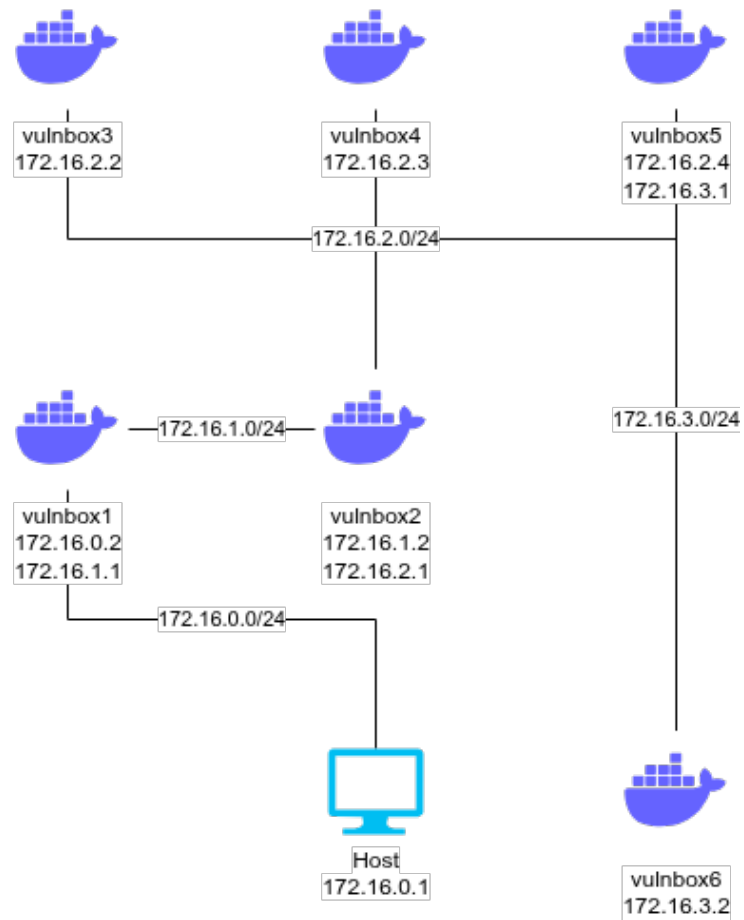


Figure 1.1: Docker network configuration

```

36 net2:
37     driver: bridge
38     driver_opts:
39         com.docker.network.bridge.name: "vulnlab2"
40     internal: true          # blocks direct host - net2 traffic
41     ipam:
42         config:
43             - subnet: 172.30.2.0/24
44               gateway: 172.30.2.254
45
46 net3:
47     driver: bridge
48     driver_opts:
49         com.docker.network.bridge.name: "vulnlab3"
50     internal: true
51     ipam:

```

```

52     config:
53     - subnet: 172.30.3.0/24
54       gateway: 172.30.3.254
55
56
57 services:
58   # --- VULNBOX 1 { Entering node -----
59   vulnbox1:
60     build:
61       context: ./network/vulnbox-1
62       dockerfile: Dockerfile
63     image: vulnbox1:latest
64     container_name: vulnbox1
65     hostname: vulnbox1
66     networks:
67       net0:
68         ipv4_address: 172.30.0.2
69       net1:
70         ipv4_address: 172.30.1.1
71     cap_add:
72     - NET_ADMIN
73     privileged: true
74     restart: unless-stopped
75
76   # --- VULNBOX 2 { Apache Struts 2.3.31 (CVE-2017-5638) ----
77   vulnbox2:
78     build:
79       context: ./network/vulnbox-2
80       dockerfile: Dockerfile
81     image: vulnbox2:latest
82     container_name: vulnbox2
83     hostname: vulnbox2
84     networks:
85       net1:
86         ipv4_address: 172.30.1.2
87       net2:
88         ipv4_address: 172.30.2.1
89     cap_add:
90     - NET_ADMIN
91     privileged: true
92     restart: unless-stopped
93
94   # --- VULNBOX 3 { DVWA -----
95   vulnbox3:
96     build:
97       context: ./network/vulnbox-3
98       dockerfile: Dockerfile
99     image: vulnbox3:latest
100    container_name: vulnbox3
101    hostname: vulnbox3
102    networks:
103      net2:
104        ipv4_address: 172.30.2.2
105    cap_add:
106    - NET_ADMIN
107    privileged: true
108    restart: unless-stopped
109
110   # --- VULNBOX 4 { SSH weak credentials -----

```

```

111 vulnbox4:
112     build:
113         context: ./network/vulnbox-4
114         dockerfile: Dockerfile
115     image: vulnbox4:latest
116     container_name: vulnbox4
117     hostname: vulnbox4
118     networks:
119         net2:
120             ipv4_address: 172.30.2.3
121     cap_add:
122         - NET_ADMIN
123     privileged: true
124     restart: unless-stopped
125
126 # --- VULNBOX 5 { Samba -----
127 vulnbox5:
128     build:
129         context: ./network/vulnbox-5
130         dockerfile: Dockerfile
131     image: vulnbox5:latest
132     container_name: vulnbox5
133     hostname: vulnbox5
134     networks:
135         net2:
136             ipv4_address: 172.30.2.4
137         net3:
138             ipv4_address: 172.30.3.1
139     cap_add:
140         - NET_ADMIN
141     privileged: true
142     restart: unless-stopped
143
144 # --- VULNBOX 6 { MySQL no root password -----
145 vulnbox6:
146     build:
147         context: ./network/vulnbox-6
148         dockerfile: Dockerfile
149     image: vulnbox6:latest
150     container_name: vulnbox6
151     hostname: vulnbox6
152     networks:
153         net3:
154             ipv4_address: 172.30.3.2
155     cap_add:
156         - NET_ADMIN
157     privileged: true
158     restart: unless-stopped

```

To ensure proper network setup it was crucial to set the correct routes through all the containers, so that they could communicate with each other in the way we wanted. To do so we wrote the following script:

```

1  #!/bin/bash
2  # Stop and remove any existing containers
3  docker compose down --remove-orphans
4  # Start the vulnbox network
5  docker compose up --remove-orphans -d
6  # Allow forwarding between containers

```

```

7 iptables -I FORWARD 1 -s 172.30.0.0/22 -d 172.30.0.0/22 -j ACCEPT
8 # Remove the default IP addresses from the bridge interfaces
9 ip addr del 172.30.1.254/24 dev vulnlab1
10 ip addr del 172.30.2.254/24 dev vulnlab2
11 ip addr del 172.30.3.254/24 dev vulnlab3
12 # Host
13 # Tell your physical machine to send lab traffic to vulnbox1
14 ip route replace 172.30.1.0/24 via 172.30.0.2
15 ip route replace 172.30.2.0/24 via 172.30.0.2
16 ip route replace 172.30.3.0/24 via 172.30.0.2
17 # vulnbox1
18 docker exec -u 0 vulnbox1 sysctl -w net.ipv4.ip_forward=1
19 docker exec -u 0 vulnbox1 ip route replace 172.30.2.0/24 via 172.30.1.2
20 docker exec -u 0 vulnbox1 ip route replace 172.30.3.0/24 via 172.30.1.2
21 docker exec -u 0 vulnbox1 ip route add default via 172.30.0.1
22 # vulnbox2
23 docker exec -u 0 vulnbox2 sysctl -w net.ipv4.ip_forward=1
24 docker exec -u 0 vulnbox2 ip route replace 172.30.3.0/24 via 172.30.2.4
25 docker exec -u 0 vulnbox2 ip route add default via 172.30.1.1
26 # vulnbox3
27 docker exec -u 0 vulnbox3 ip route add 172.30.3.0/24 via 172.30.2.4
28 docker exec -u 0 vulnbox3 ip route add default via 172.30.2.1
29 # vulnbox4
30 docker exec -u 0 vulnbox4 ip route replace 172.30.3.0/24 via 172.30.2.4
31 docker exec -u 0 vulnbox4 ip route add default via 172.30.2.1
32 # vulnbox5
33 docker exec -u 0 vulnbox5 sysctl -w net.ipv4.ip_forward=1
34 docker exec -u 0 vulnbox5 ip route add default via 172.30.2.1
35 # vulnbox6
36 docker exec -u 0 vulnbox6 ip route add default via 172.30.3.1

```

To let Greenbone/OpenVAS scan the containers, we had to create further rules. Since the VM is intended as a vulnerable machine, we had no intention to let the firewall block any traffic, therefore, firstly, we flushed all its rules and then we build up our own ruleset, which allows all the traffic from the Greenbone/OpenVAS to the container's network.

```

1 #!/bin/bash
2
3 # DO NOT RUN THIS ON YOUR HOST MACHINE
4 # VM ONLY
5
6 docker_interface="vulnlab0"
7 docker_subnet="172.30.0.0/22"
8 interface=$(ip -o -f inet addr show | awk '/scope global/ {print $2}' | head -n
↪ 1)
9
10 ./setup.sh
11 # Flush everything and start clean
12 iptables -F FORWARD
13 iptables -t nat -F POSTROUTING
14 nft flush chain ip raw PREROUTING
15
16 # Allow forwarding between containers
17 iptables -I FORWARD 1 -s $docker_subnet -d $docker_subnet -j ACCEPT
18
19 # Re-add Docker's default NAT rules
20 iptables -t nat -A POSTROUTING -s $docker_subnet ! -o $docker_interface -j
↪ MASQUERADE

```

```

21 iptables -t nat -A POSTROUTING -s 172.17.0.0/16 ! -o docker0 -j MASQUERADE
22
23 # Allow forwarding between eth and bridge
24 iptables -I FORWARD 1 -i $interface -o $docker_interface -j ACCEPT
25 iptables -I FORWARD 1 -i $docker_interface -o $interface -j ACCEPT
26
27 # Allow Docker chain
28 iptables -I DOCKER 1 -i $interface -d $docker_subnet -j ACCEPT

```

vulnbox1

The first vulnbox was intended to be vulnerable to log4Shell (CVE-2021-44228), a critical vulnerability in the Apache Log4j library. However, since the container was handmade, Greenbone/OpenVAS was not able to detect it.

```

1  # Vulnbox 1 - Log4Shell
2  # Pivot node for networking
3  FROM maven:3.9.9-eclipse-temurin-8 AS build
4
5  WORKDIR /build
6  COPY pom.xml .
7  RUN mvn -q -e -DskipTests
   ↪ org.apache.maven.plugins:maven-dependency-plugin:3.3.0:go-offline
8
9  COPY src ./src
10 RUN mvn -q -DskipTests package
11
12 # ---- runtime ----
13 FROM eclipse-temurin:8-jre-jammy
14
15 # install tools for your lab
16 RUN apt-get update -o Acquire::AllowInsecureRepositories=true \
17     -o Acquire::AllowDowngradeToInsecureRepositories=true && \
18     apt-get install -y --allow-unauthenticated iproute2 curl && \
19     apt-get clean && \
20     rm -rf /var/lib/apt/lists/* /var/cache/apt/archives/*
21
22 WORKDIR /app
23 COPY --from=build /build/target/vulnapp-1.0-jar-with-dependencies.jar .
24
25 EXPOSE 8080
26
27 CMD ["java", "-jar", "vulnapp-1.0-jar-with-dependencies.jar"]

```

Even if the vulnerability was not detected, the main role of the first vulnbox was to route all the traffic to the docker network and vice versa and, in that sense, it was successful.

vulnbox2

The second vulnbox presents a vulnerable version of the Apache Struts framework, which is affected by the CVE-2017-5638 vulnerability.

```

1  # Apache Struts 2.3.31 { CVE-2017-5638
2  # The Content-Type header is parsed by the Jakarta Multipart parser.
3  # A malformed value triggers OGNL expression evaluation → unauthenticated RCE.

```

```

4  # This is the exact vulnerability used in the 2017 Equifax breach.
5
6  FROM ubuntu:20.04
7
8  ENV DEBIAN_FRONTEND=noninteractive
9
10 # Install Java 8 + Tomcat 8 + wget
11 RUN apt-get update -qq && \
12     apt-get install -y -qq \
13         openjdk-8-jdk-headless \
14         wget \
15         curl \
16         unzip \
17         iproute2 && \
18     rm -rf /var/lib/apt/lists/*
19
20 ENV JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
21 ENV PATH="$JAVA_HOME/bin:$PATH"
22
23 # -- Tomcat 8.5 -----
24 ENV TOMCAT_VERSION=8.5.87
25 ENV CATALINA_HOME=/opt/tomcat
26
27 RUN wget -q
28     ↪ "https://archive.apache.org/dist/tomcat/tomcat-8/v${TOMCAT_VERSION}/bin/apache-tomcat-${TOMCAT_VERSION}.tar.gz" \
29     ↪ \
30         -O /tmp/tomcat.tar.gz && \
31     mkdir -p "$CATALINA_HOME" && \
32     tar xzf /tmp/tomcat.tar.gz -C "$CATALINA_HOME" --strip-components=1 && \
33     rm /tmp/tomcat.tar.gz
34
35 # -- Struts 2.3.31 showcase WAR (contains the vulnerable REST plugin) --
36 ENV STRUTS_VERSION=2.3.31
37
38 RUN wget -q
39     ↪ "https://archive.apache.org/dist/struts/${STRUTS_VERSION}/struts-${STRUTS_VERSION}-apps.zip"
40     ↪ \
41         -O /tmp/struts-apps.zip && \
42     unzip -q /tmp/struts-apps.zip
43     ↪ "struts-${STRUTS_VERSION}/apps/struts2-showcase.war" \
44     ↪ -d /tmp/struts-extract/ && \
45     cp "/tmp/struts-extract/struts-${STRUTS_VERSION}/apps/struts2-showcase.war" \
46     "${CATALINA_HOME}/webapps/struts2-showcase.war" && \
47     rm -rf /tmp/struts-apps.zip /tmp/struts-extract
48
49 # Remove default Tomcat apps to keep the attack surface clean/focused
50 RUN rm -rf \
51     "$CATALINA_HOME/webapps/ROOT" \
52     "$CATALINA_HOME/webapps/docs" \
53     "$CATALINA_HOME/webapps/examples" \
54     "$CATALINA_HOME/webapps/host-manager" \
55     "$CATALINA_HOME/webapps/manager"
56
57 # Tomcat manager creds (useful for post-exploit exploration)
58 COPY tomcat-users.xml "$CATALINA_HOME/conf/tomcat-users.xml"
59
60 EXPOSE 8080

```

```
58 CMD ["sh", "-c", "$CATALINA_HOME/bin/catalina.sh run"]
```

vulnbox3

The third vulnbox is based on DVWA (Damn Vulnerable Web Application), a web application that is intentionally vulnerable to various types of attacks. We had no clue if Greenbone/OpenVAS would have been able to detect any vulnerabilities here, but since it's in fact the scope of such experiment, we decided to include it.

```
1 # Vulnbox 3 { DVWA
2 # Damn Vulnerable Web Application (DVWA) is a PHP/MySQL web app designed for
  ↳ security training.
3 # It contains multiple vulnerabilities of varying difficulty, making it ideal for
  ↳ practice.
4
5 FROM vulnerables/web-dvwa
6
7 ENV MYSQL_PASS=p@ssw0rd
8 ENV HOSTNAME=vulnbox3
9
10
11 EXPOSE 80 443
```

vulnbox4

The fourth vulnbox is a simple Ubuntu 22.04 image with weak SSH credentials.

```
1 # Vulnbox 4 - SSH Service with Weak Credentials
2 # This container runs an SSH server with weak credentials for testing purposes.
3 FROM ubuntu:22.04
4
5 RUN apt-get update -qq && \
6     apt-get install -y -qq openssh-server iproute2 && \
7     rm -rf /var/lib/apt/lists/* && \
8     useradd -m -s /bin/bash admin; \
9     echo 'admin:admin123' | chpasswd; \
10    useradd -m -s /bin/bash user; \
11    echo 'user:password' | chpasswd; \
12    mkdir -p /run/sshd && \
13    sed -i 's/#PermitRootLogin.*/PermitRootLogin yes/' /etc/ssh/sshd_config && \
14    sed -i 's/#PasswordAuthentication.*/PasswordAuthentication yes/'
  ↳ /etc/ssh/sshd_config && \
15    echo 'root:toor' | chpasswd
16
17 EXPOSE 22
18
19
20 CMD ["/usr/sbin/sshd", "-D"]
```

vulnbox5

The fifth vulnbox is a weak Samba server.

```
1 # Vulnbox 5 - Samba (EternalBlue-era misconfiguration)
```

```

2  # This container runs a Samba server with a misconfiguration that allows for
   ↪ potential exploitation.
3
4  FROM dperson/samba
5
6  RUN mkdir -p /public /private && \
7      chmod 777 /public && \
8      chmod 700 /private
9
10 ENV USERID=1000 \
11     GROUPID=1000
12
13
14 CMD ["-u", "guest;guest", \
15     "-s", "public;/public;yes;no;yes;all;none", \
16     "-s", "private;/private;no;no;no;all;none"]

```

vulnbox6

The sixth vulnbox is a MySQL server with no password.

```

1  FROM mysql:5.7
2
3  ENV MYSQL_ALLOW_EMPTY_PASSWORD=yes
4
5  RUN yum clean all && \
6      yum install -y iproute && \
7      yum clean all && \
8      rm -rf /var/cache/yum

```

1.2.4 FTP

The FTP server is a simple instance of `vsftpd` with multiple weak configurations, such as:

```

1  local_enable=YES
2  write_enable=YES
3  pam_service_name=vsftpd

```

1.2.5 Samba

The Samba server is configured to allow insecure access to a shared folder.

```

1  #!/bin/bash
2
3  apt-get install -y samba
4
5  tee /etc/samba/smb.conf << EOF
6  [global]
7      workgroup = WORKGROUP
8      server string = Vuln Samba
9      map to guest = Bad User
10     guest account = nobody
11
12     # Weak / legacy protocol support
13     server min protocol = NT1

```

```

14     ntlm auth = yes
15     lanman auth = yes
16
17     # Disable signing
18     server signing = disabled
19
20 [vulnshare]
21     path = /srv/samba/vuln
22     browsable = yes
23     writable = yes
24     guest ok = yes
25     read only = no
26     force user = nobody
27 EOF
28
29 mkdir -p /srv/samba/vuln
30 chmod 777 /srv/samba/vuln
31 systemctl enable --now smb

```

1.2.6 SSH

The SSH server is configured with weak configurations, such as:

```

1 PermitRootLogin yes
2 PasswordAuthentication no

```

1.2.7 Syslog

The Syslog server is configured to allow remote logging without authentication on port 514.

```

1  #!/bin/bash
2
3  #apt-get update
4  #apt-get install -y rsyslog
5  #apt-get clean
6  #rm -rf /var/lib/apt/lists/*
7
8  # Enable UDP + TCP syslog reception
9  cat << EOF > /etc/rsyslog.d/10-remote.conf
10 # Load modules
11 module(load="imudp")
12 input(type="imudp" port="514")
13
14 module(load="imtcp")
15 input(type="imtcp" port="514")
16
17 # Log EVERYTHING from remote hosts
18 *.* /var/log/remote.log
19
20 # No rate limiting (intentionally bad)
21 \$$SystemLogRateLimitInterval 0
22 EOF
23
24 # Ensure rsyslog is enabled
25 systemctl enable rsyslog
26 systemctl restart rsyslog

```

```

27
28 mkdir -p /var/log/
29 chmod 777 -R /var/log/

```

1.3 Windows XP

Windows XP is an old operating system that is no longer supported by Microsoft, which makes it vulnerable to various types of attacks. Just its mere presence in the network is a vulnerability, since it can be easily exploited by attackers. To set up Windows XP, we just installed its virtual machine on VirtualBox and connected it to the same network of the other machines.

1.4 Network configuration

The whole network configuration is an Host-only virtual network created with VirtualBox, which allows all the machines to communicate with each other and have no access to other networks, such as the Internet.

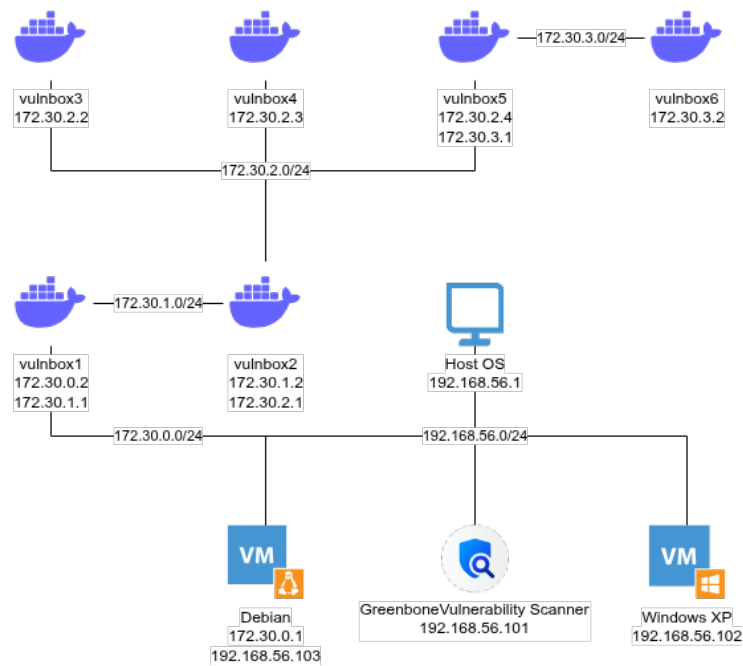
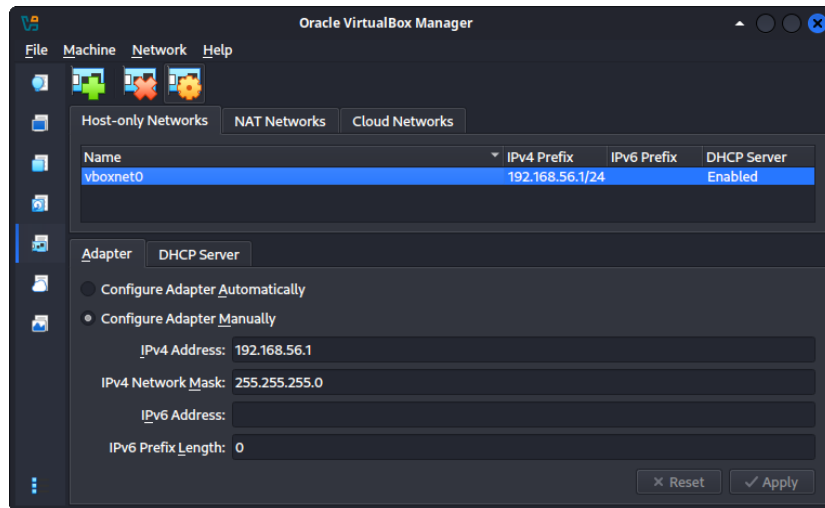


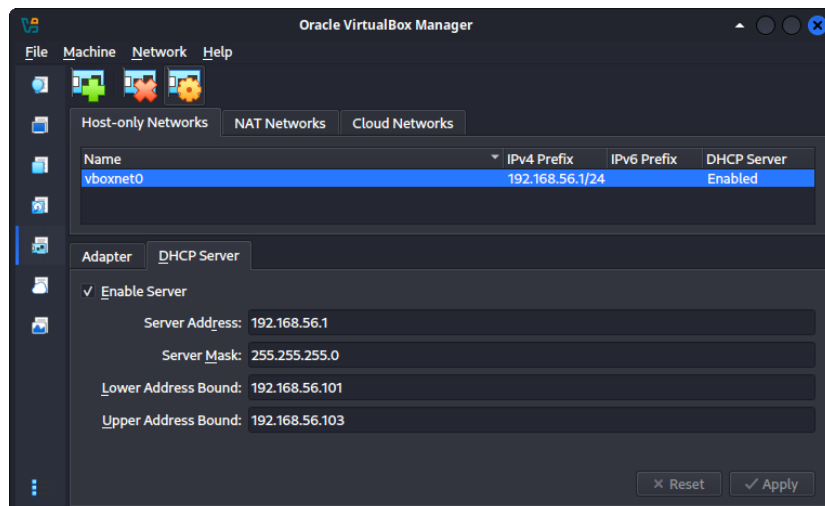
Figure 1.2: Full laboratory's network

In order to create the virtual network and connect all the machines to it, we followed the following procedure:

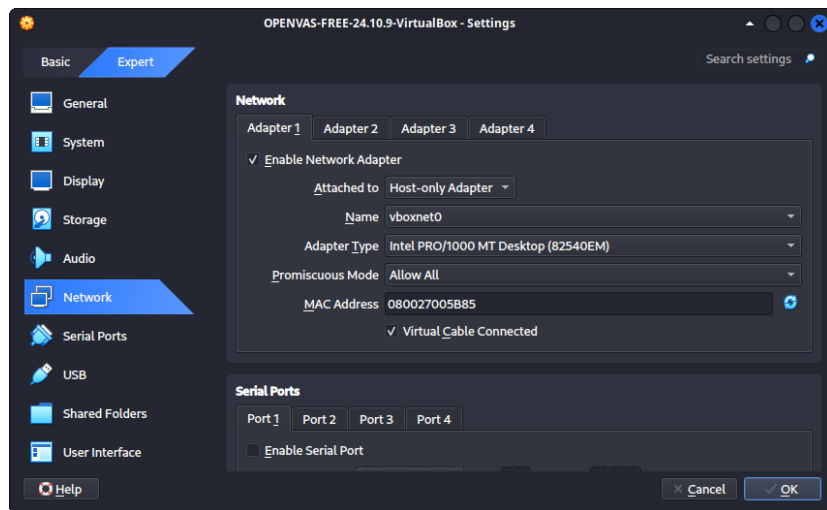
1. Create a Host-only network in VirtualBox



2. Enable the DHCP server for the Host-only network enough to assign IP addresses to all the 3 machines



3. Go to settings of each machine and connect it to the Host-only network



Chapter 2

System Discovery

In this chapter we will describe how to perform a System Discovery using OpenVAS. There must be clarified that this type of scan is not a Vulnerability Scan, it's just a detection of the targets of a system, their open ports, the services running on them and its whole topology.

2.1 Get Debian VM IP address

Before performing the System Discovery, we need to know the IP address of the Debian VM, since we need to specify it as Greenbone default gateway to reach the docker network where the vulnboxes are located. Open the Debian VM, which credentials are:

- Username: root
- Password: root

Then go to the folder `/network-security/project/docker-vulnet/` and run the command `bash hardcore-setup.sh` to burn down the firewall and build up some rules which allow the Debian machine to redirect the traffic to the docker network.

```
Debian GNU/Linux 13 debian tty1
debian login: root
Password:
Linux debian 6.12.74+deb13+1-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.74-2 (2026-03-08) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@debian:~# cd network-security/project/docker-vulnet/
root@debian:~/network-security/project/docker-vulnet# bash hardcore-setup.sh
[+] down 10/10
+ Container vulnbox6 Removed
+ Container vulnbox1 Removed
+ Container vulnbox2 Removed
+ Container vulnbox3 Removed
+ Container vulnbox4 Removed
+ Container vulnbox5 Removed
+ Network docker-vulnet_net1 Removed
+ Network docker-vulnet_net2 Removed
+ Network docker-vulnet_net3 Removed
+ Network docker-vulnet_net0 Removed
[+] up 10/10
```

Figure 2.1: Debian VM - Initial setup

After that, run the command `ip addr | grep 192` to get the IP address of the Debian VM.

```
root@debian:~# ip a | grep 192
    inet 192.168.56.103/24 brd 192.168.56.255 scope global dynamic noprefixroute enp0s3
root@debian:~# _
```

Figure 2.2: Debian VM - IP address

2.2 Configure Greenbone Gateway

Before performing the System Discovery, we need to configure the Greenbone Gateway to reach the docker network where the vulnboxes are located. Open the OpenVAS VM, which credentials are:

- Username: admin
- Password: admin

Go to **Setup** > **Network** > **Global Gateway** and change the default gateway according to the IP address of the Debian VM, which we got in the previous section. Then save the configuration.



Figure 2.3: Greenbone VM - Gateway configuration

2.3 Greenbone Web Interface

Open the *About* section in the main menu of the Greenbone VM.

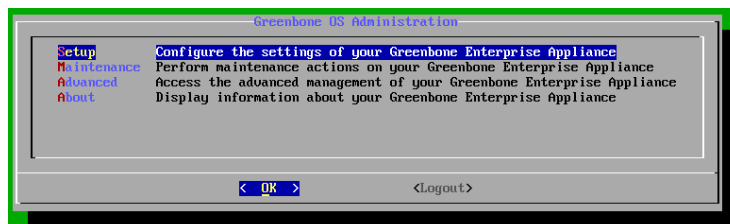


Figure 2.4: Greenbone VM - Main section

Check out the IP address of the VM.

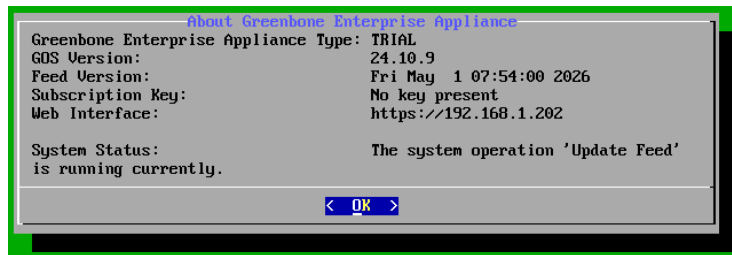


Figure 2.5: Greenbone VM - About section

Open a web browser and type the URL for the web interface in the search bar.

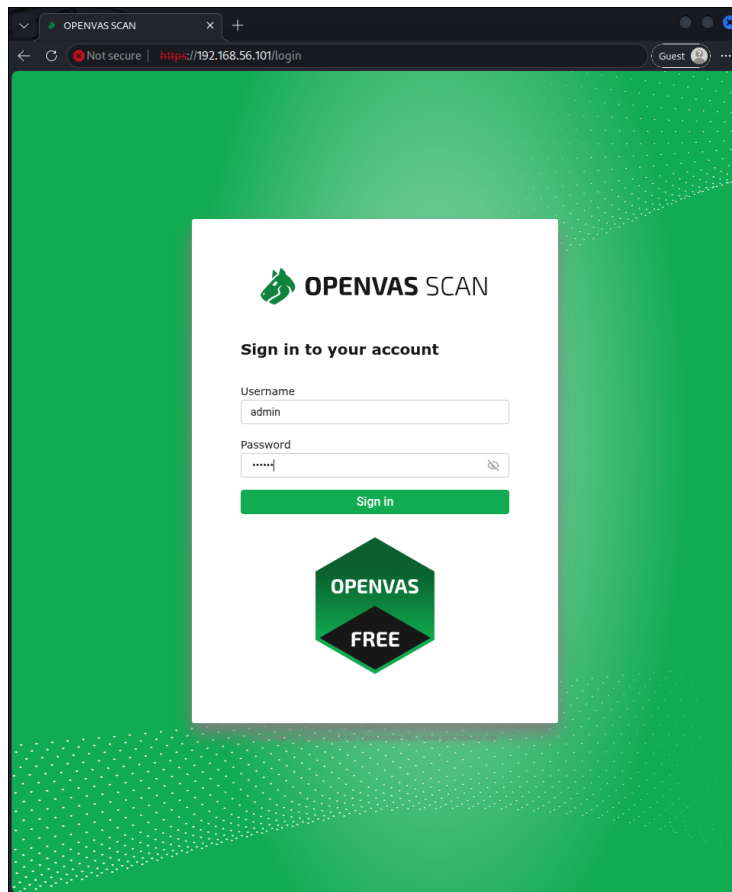


Figure 2.6: Greenbone Web Interface - Login page

Log in with the credentials:

- Username: admin
- Password: admin

You will see the main page of the web interface.

2.4 System Scan

Now go to the *Scans* section and click on *Tasks*.

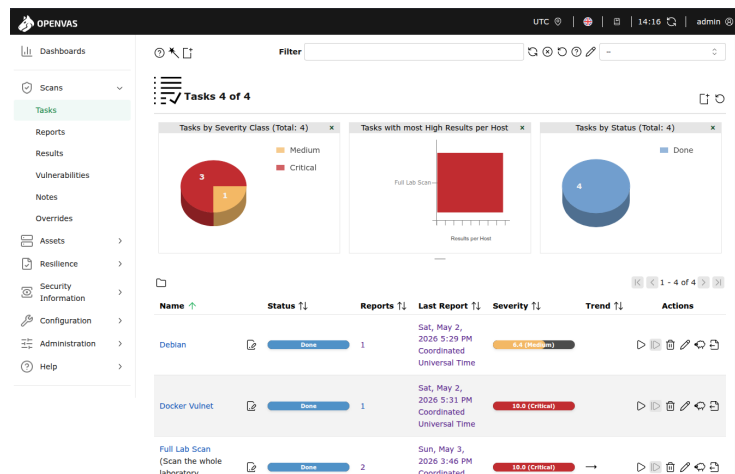


Figure 2.7: Greenbone Web Interface - Main page

Modify the *Docker Vulnet* task. Edit it by choosing *System Discovery* as the scan configuration, then save the task.

Edit Task Docker Vulnet

Scan targets

Vulnet Target

Add results to Assets

☒ Yes ☐ No

Apply Overrides

☒ Yes ☐ No

Min QoD

70

Alterable Task

☐ Yes ☐ No

Auto Delete Reports

☒ Do not automatically delete reports

☐ Automatically delete oldest reports but always keep newest

5

reports

Scanner

OpenVAS Default

Scan Config

System Discovery

Order for target hosts

Sequential

Maximum concurrently executed NVTs per host

7

Maximum concurrently scanned hosts

7

Cancel

Save

Figure 2.8: Greenbone Web Interface - Task modification

Now we can launch the task pressing the *Play* button.



Figure 2.9: Greenbone Web Interface - Task launch



Figure 2.10: Greenbone Web Interface - Task progress

After about 10 minutes, the task will be completed and we can see the results. To see the results, click on the task and then on the *Report* tab. In the upper right corner, click on the *Remove filters* button to see all the results. This is required because by default the web interface shows only the results with a severity score, but since the *System Discovery* scan configuration doesn't detect vulnerabilities, all the results have a severity score of 0, so we need to remove the filters to see them all.



Figure 2.11: Greenbone Web Interface - Remove filters

Now we can see all the results of the scan which comprehends:

- The detected hosts in the given IP range
- The open ports on the hosts
- The services running on the open ports
- The topology of the system

Even if the scanned system only has 6 vulnboxes, as we can see, OpenVAS detected a lot more hosts, since in the OpenVAS view, each IP address corresponds to a host, and in the scanned system some vulnboxes have more than one IP address, therefore OpenVAS detected all of them as distinct hosts.

[illegible]

Figure 2.12: Greenbone Web Interface - Scan results

Operating System 	CPE 	Hosts 	Severity 
 Linux Kernel	cpe:/o:linux:kernel	7	0.0 (Log)
 Debian GNU/Linux 9	cpe:/o:debian:debian_linux:9	1	0.0 (Log)
 Debian GNU/Linux 13	cpe:/o:debian:debian_linux:13	1	0.0 (Log)
 Ubuntu 22.04	cpe:/o:canonical:ubuntu_linux:22.04	1	0.0 (Log)

Figure 2.13: Greenbone Web Interface - Operating Systems

The results of this scan can be also be seen in the *Assests_Hosts* section of the web interface, where we can see all the detected hosts and their open ports and services.

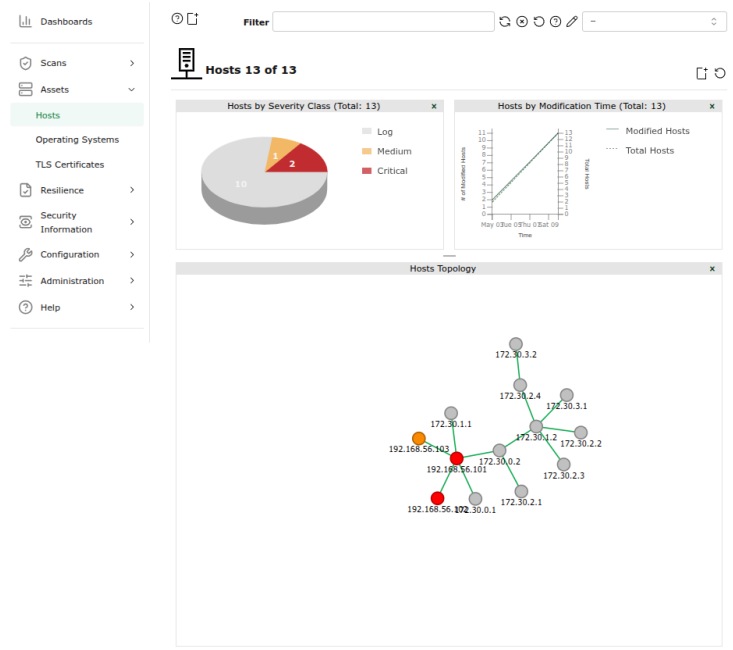


Figure 2.14: Greenbone Web Interface - Hosts section

We can here check if the detected topology of the system is correct, by comparing it with the one we designed.

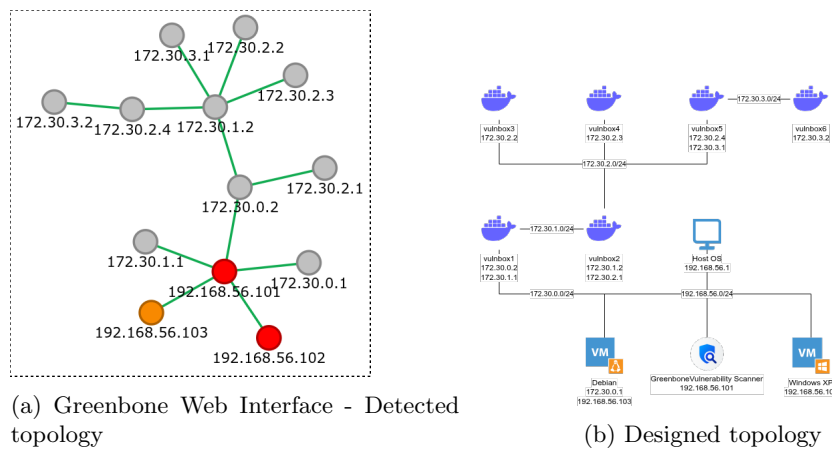


Figure 2.15: Greenbone Web Interface - Detected topology vs Designed topology

As we can see, the detected topology is correct, even if there are more hosts than vulnboxes, the overall topology is connected in the right way.

Chapter 3

OpenVAS Scan

3.1 OpenVAS Scan

In this section it will be explained how to start a scan (that could be just for an host, or a group of hosts). We will create a target, which means declaring which host (or group of hosts) to be scanned, which port range and protocol.

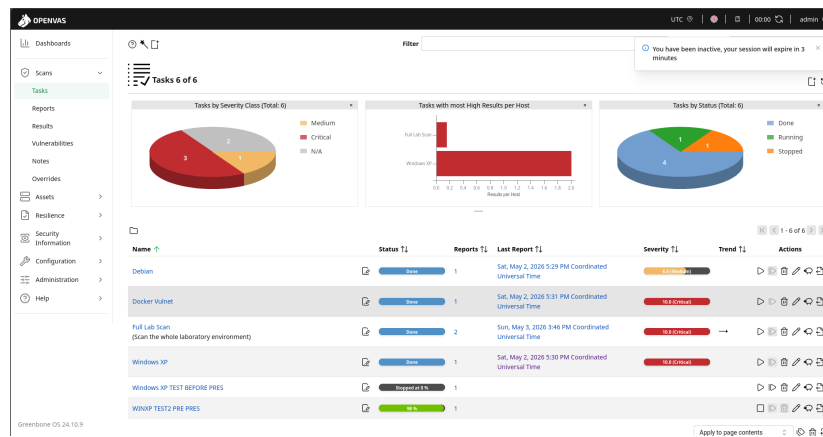
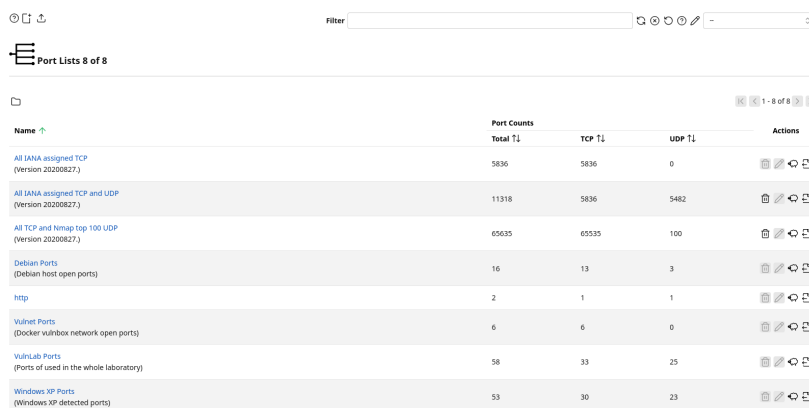


Figure 3.1: Scan windows

This is the scan windows, where you will create the tasks which are configurations for the targets and ports that are going to be scanned, how many NVTs (Network Vulnerability Test) are going to be used for the scan, how many concurrent hosts are being scanned at a given moment etc. We will get to all of the configuration in time.

3.1.1 Configuration → Ports



Port Lists 8 of 8

Name	Port Counts			Actions
	Total T1	TCP T1	UDP T1	
All IANA assigned TCP (Version 20200827)	5836	5836	0	[Add] [Edit] [Delete]
All IANA assigned TCP and UDP (Version 20200827)	11318	5836	5482	[Add] [Edit] [Delete]
All TCP and Nmap top 100 UDP (Version 20200827)	65635	65635	100	[Add] [Edit] [Delete]
Debian Ports (Debian host open ports)	16	13	3	[Add] [Edit] [Delete]
http	2	1	1	[Add] [Edit] [Delete]
Vulnet Ports (Docker vulnbox network open ports)	6	6	0	[Add] [Edit] [Delete]
VulnLab Ports (Ports of used in the whole laboratory)	58	33	25	[Add] [Edit] [Delete]
Windows XP Ports (Windows XP detected ports)	53	30	23	[Add] [Edit] [Delete]

Figure 3.2: Port creation interface

This is the ports creation interface, you can create custom ports ranges looking for services that use TCP/UDP. For the most part OpenVAS works by giving us templates, in this case are for a complete scans of all IANA (Internet Assigned Numbers Authority) ports, which means it will scan for all 65536 ports only TCP, UDP, both etc. It really just depends on which preset is chosen.

OpenVAS encodes ports in a format like T:1 – 1023, 4500,U:2 – 100, 3246 which means that it will scan for TCP ports between 1 and 1023 and 4500 alone, and for UDP ports from 2 to 100 and 3246 alone. In this way you can define ports range and choose to do only TCP or UDP scans.

Under the hood OpenVAS uses Nmap to scan the ports, and just like it to scan UDP ports it will take considerably longer than TCP. This is because when a TCP request is made, the server either accept the connection or it rejects it, while for UDP being a connection-less protocol will wait until timeout. So beware about this aspect if you don't want the scan to take hours.

For our experiments we will create two presets: one for ftp service (port 21) and one for an http server (port 80). Go on the add port icon on the top of the interface (the little square with a plus), and you should get this popup.

New Port List ✕

Name: WIN XP ONLY FTP

Comment:

Port Ranges: ☒ Manual T:21 ☐ From file

Cancel Save

New Port List ✕

Name: WIN XP FTP&HTTP

Comment:

Port Ranges: ☒ Manual T:21,80 ☐ From file

Cancel Save

(a) Port configuration limited to TCP 21 (FTP). (b) Port configuration limited to TCP 21 and 80 (FTP).

Figure 3.3: Ports configuration

Those configurations are going to be used later to scan a Windows XP target with FTP and HTTP servers.

Credentials for target access

The screenshot shows the OpenVAS web interface with the 'Credentials' tab selected. A table lists 10 credentials, each with a Name, Type, Allow insecure use, Login, and Actions column. The credentials include Admin, Admin vulnbox, Debian, Guest, Manager, MySQL vulnbox, PostgreSQL, Root, Tomcat, and User.

Name	Type	Allow insecure use	Login	Actions
Admin	Username + Password	Yes	admin	[Edit] [Delete] [Add]
Admin vulnbox	Username + Password	Yes	admin	[Edit] [Delete] [Add]
Debian	Username + Password	Yes	debian	[Edit] [Delete] [Add]
Guest	Username + Password	Yes	guest	[Edit] [Delete] [Add]
Manager	Username + Password	Yes	manager	[Edit] [Delete] [Add]
MySQL vulnbox (vulnbox:mysql password)	Username + Password	Yes	vulnbox	[Edit] [Delete] [Add]
Postgres	Username + Password	Yes	postgres	[Edit] [Delete] [Add]
Root	Username + Password	Yes	root	[Edit] [Delete] [Add]
Tomcat	Username + Password	Yes	tomcat	[Edit] [Delete] [Add]
User	Username + Password	Yes	user	[Edit] [Delete] [Add]

Figure 3.4: Credentials window

Credentials are used for deeper scans, you give the credentials to OpenVAS and it will try to login to the machine, scanning the host from within, looking at configuration files, installed packages etc. This is done to improve scan performance since it flags problems that are not visible from the outside, but may lead to exploitable vulnerabilities. This function can also be used to check for default credentials if they are in use or no.

New Credential

×

Name

Test

Comment

Type

Username + Password

Allow insecure use

☐ Yes
☒ No

Auto-generate

☐ Yes
☒ No

Username

Password

Cancel

Save

New Credential

×

Name

Test

Comment

Type

Username + Password

Username + SSH Key

SNMP

S/MIME Certificate

PGP Encryption Key

Password only

Client Certificate

Cancel

Save

(a) FTP Target Step 1

(b) FTP Target Step 2

Figure 3.5: Configuration for the FTP only target

As you can see many possibilities are available, setting both username and password, ssh access, certificates etc. But for our example this feature will not be used.

3.1.2 Target configuration

Dashboards

Scans

Assets

Resilience

Security Information

Configuration

Targets

Port Lists

Credentials

Scan Configs

Report Formats

Scanners

Filters

Tags

Administration

Help

Targets 13 of 13

Name	Hosts T1	IPs T1	Port List T1	Credentials	Actions
Debian Target (Debian VM vulnbox)	192.168.56.103	1	Debian Ports	SSHDebian SSH Elevate-Root SMB (NTLML)Debian	
VM Targets (VMs in the laboratory environment)	192.168.56.101-192.168.56.104	4	VulnLab Ports	SSHDebian SSH Elevate-Root SMB (NTLML)Debian	
vulnbox1 (Log4Shell)	172.30.0.2	1	VulnLab Ports	SSHAdmin SSH Elevate-Root	
vulnbox2 (Apache Struts)	172.30.1.2	1	VulnLab Ports	SSHTomcat SSH Elevate-Root	
vulnbox3 (CVE44)	172.30.2.2	1	VulnLab Ports	SSHMySQL_vulnbox3 SSH Elevate-Root	
vulnbox4 (SSRF)	172.30.2.3	1	VulnLab Ports	SSHUser SSH Elevate-Admin	
vulnbox5 (Samba)	172.30.2.4	1	VulnLab Ports	SSHGuest SSH Elevate-Root SMB (NTLML)Guest	

Figure 3.6: Target configuration interface

This is the target creation interface, which is used to create all targets that are going to be scanned expressed in single IP or a range of IPs (e.g. 192.168.56.101-192.168.56.105).

Before scanning the Windows XP server you would want to go to the terminal of the machine and type **ipconfig /all**, copy the IP (which should be along the lines of 192.168.56.10X).

```

C:\Documents and Settings\windows>ipconfig /all

Windows IP Configuration

Host Name . . . . . : windows-xp
Primary Dns Suffix . . . . . :
Node Type . . . . . : Unknown
IP Routing Enabled. . . . . : No
WINS Proxy Enabled. . . . . : No

Ethernet adapter Local Area Connection:

Connection-specific DNS Suffix . :
Description . . . . . : Intel(R) PRO/1000 T Server Adapter
Physical Address. . . . . : 08-00-27-BB-85-A1
Dhcp Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes
IP Address. . . . . : 192.168.56.104
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . :
DHCP Server . . . . . : 192.168.56.100
Lease Obtained. . . . . : Tuesday, May 12, 2026 2:35:15 PM
Lease Expires . . . . . : Tuesday, May 12, 2026 2:45:15 PM

C:\Documents and Settings\windows>

```

Figure 3.7: Terminal output of ipconfig command

After the IP was discovered, its time to configure the target like so:

New Target

Name: Windows XP FTP TARGET

Comment:

Hosts: ☒ Manual 192.168.56.104

Exclude Hosts: ☒ Manual

Port List: WIN XP ONLY FTP

Alive Test: ☒ Use Scan Config Default

Cancel Save

New Target

WIN XP ONLY FTP

Alive Test: ☒ Use Scan Config Default

Credentials for authenticated checks:

SSH: Select a Credential on port 22

SMB (Kerberos): Select a Credential

SMB (NTLM): Select a Credential

ESXI: Select a Credential

SNMP: Select a Credential

Reverse Lookup Only: ☒ No

Reverse Lookup Unify: ☒ No

Cancel Save

(a) FTP Target Step 1

(b) FTP Target Step 2

Figure 3.8: Configuration for the FTP only target

In this popup you will set target name, the IP (or range of IPs), the excluded hosts in case you have a range of IPs and some host are not to be scanned, the port list preset made before and the credentials presets that are set in the credentials window (not used in this example).

Now configure the targets also for the FTP and HTTP target like so:

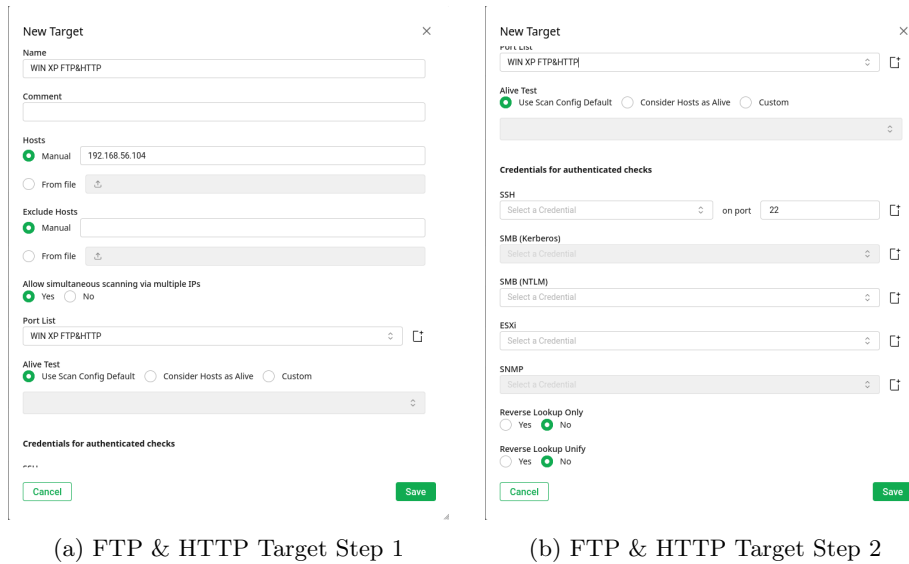


Figure 3.9: Configuration for the FTP & HTTP target

Scan configs

Name	Family	Total T1	Trend T1	NVTs	Trend T1	Actions
Base (Basic configuration template with a minimum set of NVTs required for a scan. Version 20200827)	2	→	3	→		
Discovery (Network Discovery scan configuration. Version 20201215)	11	→	320	→		
Empty (Empty and static configuration template. Version 20201215)	0	→	0	→		
Full and Fast (Most NVTs, optimized by using previously collected information. Version 20201215)	60	→	176180	→		
Full and Fast Clone 1 (Most NVTs, optimized by using previously collected information. Version 20201215)	42	→	15650	→		
Full and Fast Clone 2 (Most NVTs, optimized by using previously collected information. Version 20201215)	42	→	15650	→		
Host Discovery (Network Host Discovery scan configuration. Version 20201215)	2	→	2	→		
Host Discovery Clone 1 (Network Host Discovery scan configuration. Version 20201215)	2	→	2	→		
Log2Mail (Configuration with checks for Log2Mail and CVE-2021-46208. Version 20211227)	10	→	29	→		

Figure 3.10: Scan configs window

The scan configuration is a configuration used during the scan, essentially it tells OpenVAS how to scan and what to scan. Essentially the Full and Fast preset looks at everything possible from Cisco appliances, Fedora security, Windows services etc. This is the reason why the database is so big, occupying tens of gigabytes. Since there are thousands of settings if we look at every possible configuration, what we will do is to configure a pre-set at a high level tailoring it for Windows XP.

New Scan Config

×

Name

Windows XP Taylored Scan

Comment

Base

☐ Base with a minimum set of NVTs
 ☒ Empty, static and fast
 ☐ Full and fast

CVE

Cancel

Save

Figure 3.11: Taylored scan creation

To create it select the Empty config, save it and then edit it by clicking to the pencil icon, select the following:

Edit Scan Config Windows XP Taylored Scan

×

Name

Windows XP Taylored Scan

Comment

Empty and static configuration template. Version 20201215.

Search

Search for families, preferences, or NVTs

Edit Network Vulnerability Test Families (60)

⊞

Family	NVTs selected	Trend	Select all NVTs	Actions
AIX Local Security Checks	0 of 1	⊞ ⊞ ⊞ ⊞	☒	
Amazon Linux Local Security Checks	0 of 748	⊞ ⊞ ⊞ ⊞	☐	
Brute force attacks	0 of 9	⊞ ⊞ ⊞ ⊞	☐	
Buffer overflow	0 of 682	⊞ ⊞ ⊞ ⊞	☐	
CISCO	0 of 651	⊞ ⊞ ⊞ ⊞	☐	
CentOS Local Security Checks	0 of 3255	⊞ ⊞ ⊞ ⊞	☐	
Citrix XenServer Local Security Checks	0 of 30	⊞ ⊞ ⊞ ⊞	☐	
Compliance	0 of 16	⊞ ⊞ ⊞ ⊞	☐	

Cancel

Save

Edit Scan Config Windows XP Taylored Scan

×

CISCO

0 of 651

⊞ ⊞ ⊞ ⊞

☐

CentOS Local Security Checks

0 of 3255

⊞ ⊞ ⊞ ⊞

☐

Citrix XenServer Local Security Checks

0 of 30

⊞ ⊞ ⊞ ⊞

☐

Compliance

0 of 16

⊞ ⊞ ⊞ ⊞

☐

Databases

0 of 1200

⊞ ⊞ ⊞ ⊞

☐

Debian Local Security Checks

0 of 18192

⊞ ⊞ ⊞ ⊞

☐

Default Accounts

0 of 315

⊞ ⊞ ⊞ ⊞

☒

Denial of Service

0 of 2404

⊞ ⊞ ⊞ ⊞

☐

FTP Local Security Checks

0 of 125

⊞ ⊞ ⊞ ⊞

☒

FTP

0 of 176

⊞ ⊞ ⊞ ⊞

☒

Fedora Local Security Checks

0 of 30209

⊞ ⊞ ⊞ ⊞

☐

FortiOS Local Security Checks

0 of 36

⊞ ⊞ ⊞ ⊞

☐

FreeBSD Local Security Checks

0 of 2009

⊞ ⊞ ⊞ ⊞

☐

Gain a shell remotely

0 of 109

⊞ ⊞ ⊞ ⊞

☐

General

0 of 8619

⊞ ⊞ ⊞ ⊞

☐

Gentoo Local Security Checks

0 of 2191

⊞ ⊞ ⊞ ⊞

☐

Cancel

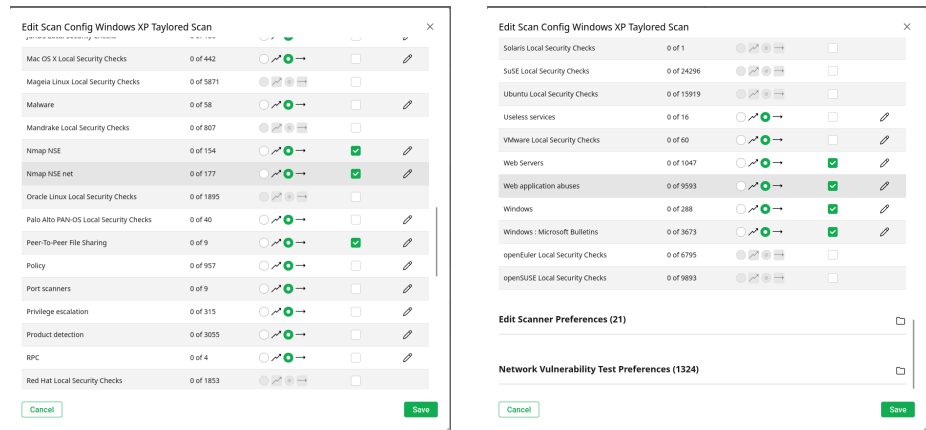
Save

(a) FTP & HTTP Taylored scan options

1

(b) FTP & HTTP Taylored scan options

2

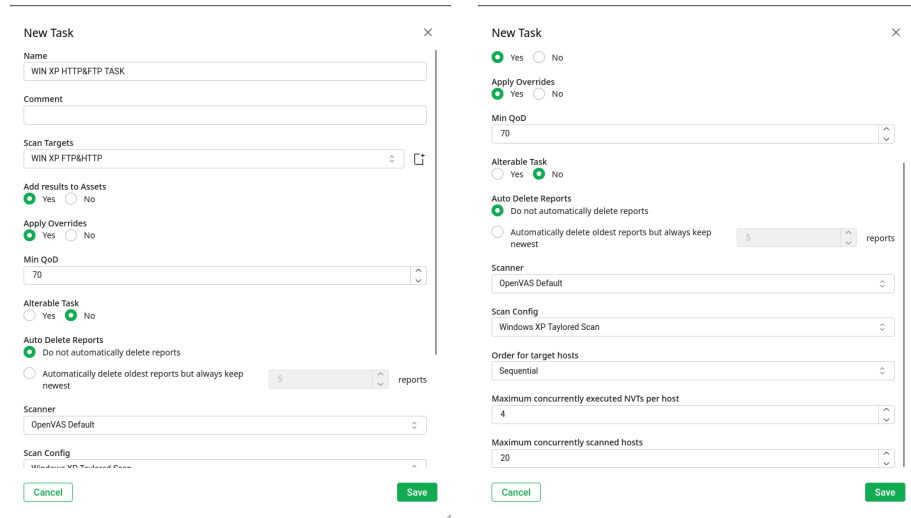


(a) FTP & HTTP Tailored scan options 3 (b) FTP & HTTP Tailored scan options 4

Figure 3.13: Configuration for the FTP & HTTP tailored scan

3.1.3 Starting the scan

Now we are ready to begin the scan, to do this go back to the tasks windows, click on the task add button and do the following:



(a) FTP & HTTP Tailored task 1 (b) FTP & HTTP Tailored task 2

Chapter 4

AI Usage Declaration

This report was prepared with the assistance of AI tools for drafting and editing support. The structure and the core contents of the report were designed by the author. All final content was reviewed and approved by the author.